



McStasScript Python interface

ISIS 2025 McStas School

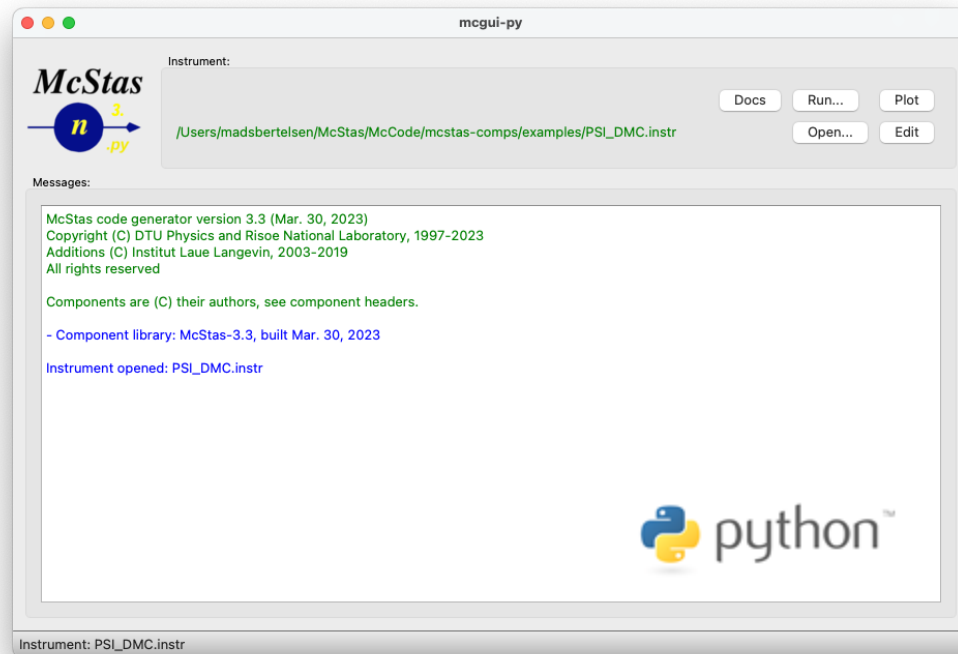
PRESENTED BY MADS BERTELSEN

2025-11-05

McStas

User interface

- C meta-language instrument file
- Help and plotting from mcgui



McStas instrument file

```
DEFINE INSTRUMENT example(theta)
```

```
DECLARE
```

```
// C-code – declare variables
```

```
double two_theta;
```

```
INITIALIZE
```

```
// C-code - perform initial calculations
```

```
two_theta = 2.0*theta;
```

```
TRACE
```

```
// McStas component sequence
```

```
Monochromator = Monochromator_flat(  
  yheight=0.1, zdepth=0.05, Q=1.78)
```

```
AT (0,0,2.0) RELATIVE source
```

```
ROTATED (0, theta, 0) RELATIVE source
```

```
FINALLY
```

```
// C-code – perform calculations after simulation
```



McStas

User interface



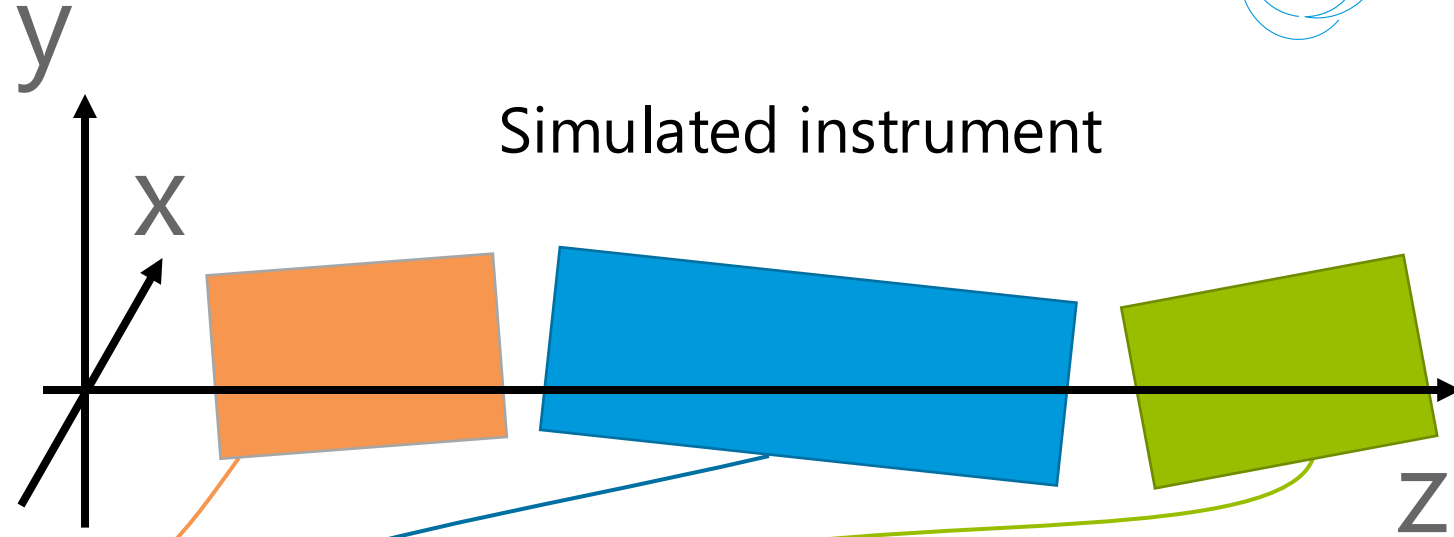
Instrument file

COMPONENT A(par=2)
AT (0, 0, 0.5)
ROTATED (-10, 0, 0)

COMPONENT B(width=0.3)
AT (0, 0, 4.0)
ROTATED (10, 0, 0)

COMPONENT C(length=2.5)
AT (0, 0, 11.5)
ROTATED (-12, 0, 0)

Simulated instrument

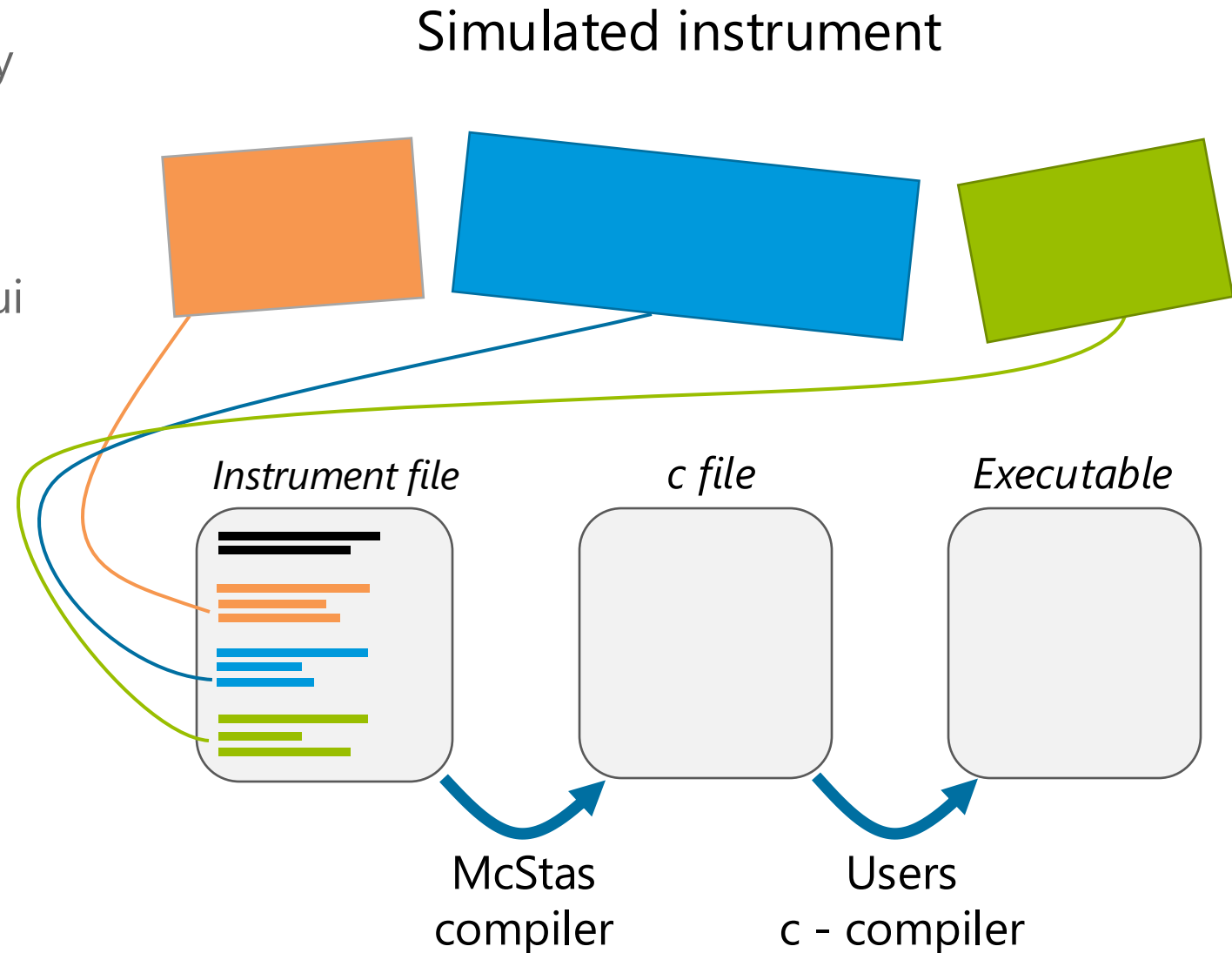


McStas



Execution of simulation

- Assemble from component library
- Place components in a sequence
- Place them in simulated space
- Execute through terminal or mcgui

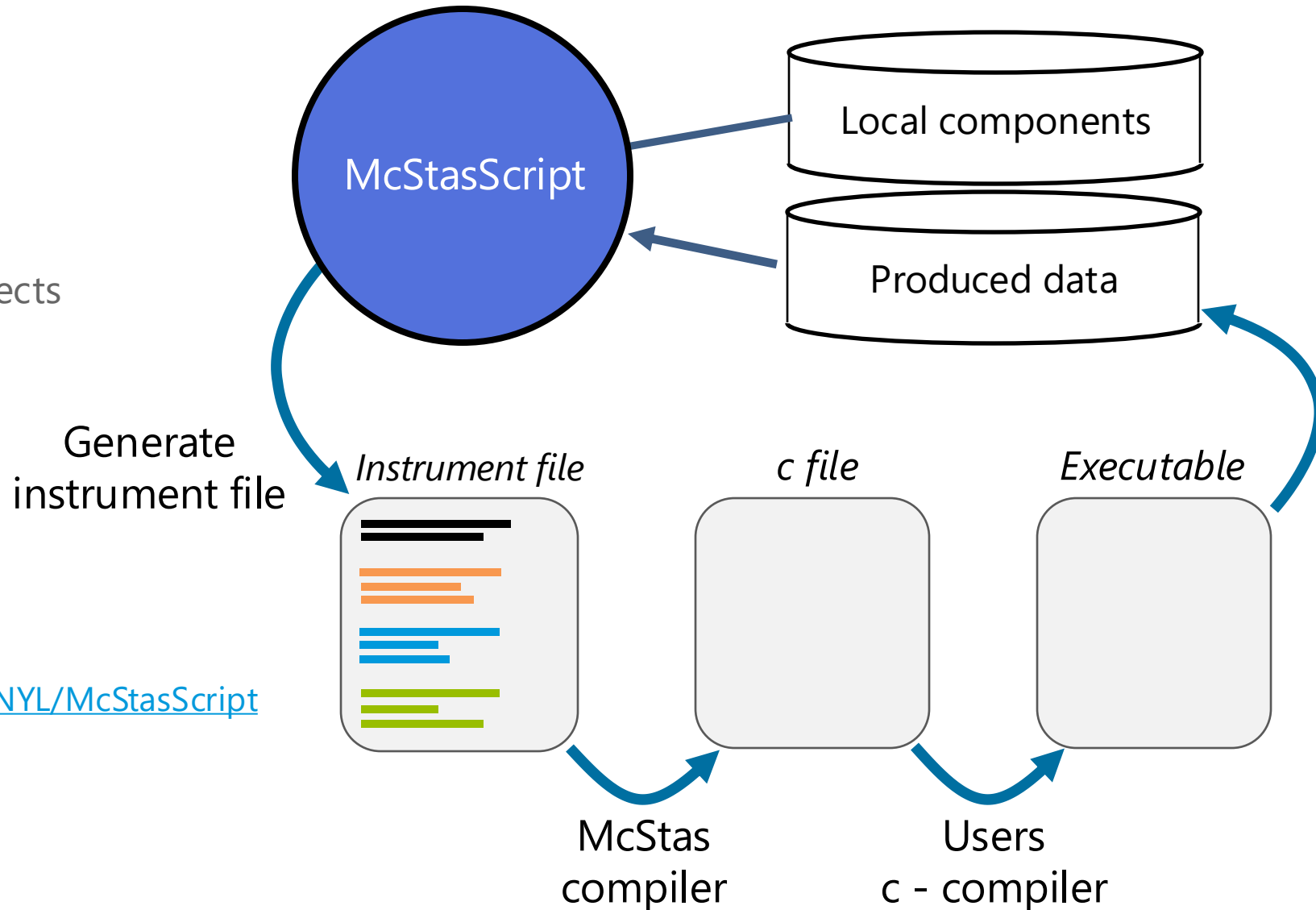


McStas

McStasScript overview

- Python McStas API
 - Build instrument object
 - Execute simulation
 - Receive data as Python objects

- Open source
- <https://github.com/PaNOOSC-ViNYL/McStasScript>
- Comprehensive test suite



McStasScript

Demo



- Instrument object
- Knows components

```
import mcstasscript as ms
```

```
instrument = ms.McStas_instr("demo_instrument")
```

```
instrument.component_help("Source_div")
```

```
____ Help Source_div _____  
|optional parameter|required parameter|default value|user specified value|  
xwidth [m] // Width of source  
yheight [m] // Height of source  
focus_aw [deg] // FWHM (Gaussian) or maximal (uniform) horz. width divergence  
focus_ah [deg] // FWHM (Gaussian) or maximal (uniform) vert. height divergence  
E0 = 0.0 [meV] // Mean energy of neutrons.  
dE = 0.0 [meV] // Energy half spread of neutrons.  
lambda0 = 0.0 [Ang] // Mean wavelength of neutrons (only relevant for E0=0)  
dlambda = 0.0 [Ang] // Wavelength half spread of neutrons.  
gauss = 0.0 [0|1] // Criterion  
flux = 1.0 [1/(s cm 2 st energy_unit)] // flux per energy unit, Angs or meV  
-----
```

McStasScript

Demo



- Component object

```
src = instrument.add_component("source", "Source_div")
src.set_parameters(xwidth= 0.1, focus_aw=1.2, focus_ah=2.3,
                  lambda0=4.0, dlambda=0.1)

print(src)
```

- Spots errors

```
COMPONENT source = Source_div(
  xwidth = 0.1, // [m]
  yheight : Required parameter not yet specified
  focus_aw = 1.2, // [deg]
  focus_ah = 2.3, // [deg]
  lambda0 = 4.0, // [Ang]
  dlambda = 0.1 // [Ang]
)
AT (0, 0, 0) ABSOLUTE
```

- Autocompletion

```
src.yheight= 0.05
print(src)

COMPONENT source = Source_div(
  xwidth = 0.1, // [m]
  yheight = 0.05, // [m]
  focus_aw = 1.2, // [deg]
  focus_ah = 2.3, // [deg]
  lambda0 = 4.0, // [Ang]
  dlambda = 0.1 // [Ang]
)
AT (0, 0, 0) ABSOLUTE
```

McStasScript

Demo



- Parameter objects

```
src.lambda0 = instrument.add_parameter("wavelength", value=5.0, comment="Wavelength [AA]")
src.dlambda = "0.1*wavelength"
print(src)
```

- Calculations

```
COMPONENT source = Source_div(
    xwidth = 0.1, // [m]
    yheight = 0.05, // [m]
    focus_aw = 1.2, // [deg]
    focus_ah = 2.3, // [deg]
    lambda0 = wavelength, // [Ang]
    dlambda = 0.1*wavelength // [Ang]
)
AT (0, 0, 0) ABSOLUTE
```

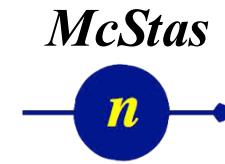
- Parameter overview

```
instrument.show_parameters()

wavelength = 5.0 // Wavelength [AA]
```


McStasScript

Demo



- Set position

```
guide = instrument.add_component("extraction", "Guide_gravity")
guide.set_parameters(w1=0.05, h1=0.05, l=4.0, m=3.5, G=-9.82)
guide.set_AT([0,0,2], RELATIVE=src)
```

- For-loop

```
last_element = guide
for index in range(1, 6):
    guide = instrument.add_component("guide_" + str(index), "Guide_gravity")
    guide.set_parameters(w1=0.05, h1=0.05, l=1.0, m=3.5, G=-9.82)
    guide.set_AT(last_element.l + 1E-3, RELATIVE=last_element)
    guide.set_ROTATED([0, 0.3, 0], RELATIVE=last_element)

    last_element = guide
```

- Component list

```
instrument.show_components()
```

source	Source_div	AT	(0, 0, 0)	ABSOLUTE	
extraction	Guide_gravity	AT	(0, 0, 2)	RELATIVE	source
guide_1	Guide_gravity	AT	(0, 0, 4.001)	RELATIVE	extraction
		ROTATED	(0, 0.3, 0)	RELATIVE	extraction
guide_2	Guide_gravity	AT	(0, 0, 1.001)	RELATIVE	guide_1
		ROTATED	(0, 0.3, 0)	RELATIVE	guide_1
guide_3	Guide_gravity	AT	(0, 0, 1.001)	RELATIVE	guide_2
		ROTATED	(0, 0.3, 0)	RELATIVE	guide_2
guide_4	Guide_gravity	AT	(0, 0, 1.001)	RELATIVE	guide_3
		ROTATED	(0, 0.3, 0)	RELATIVE	guide_3
guide_5	Guide_gravity	AT	(0, 0, 1.001)	RELATIVE	guide_4
		ROTATED	(0, 0.3, 0)	RELATIVE	guide_4

McStasScript

Demo

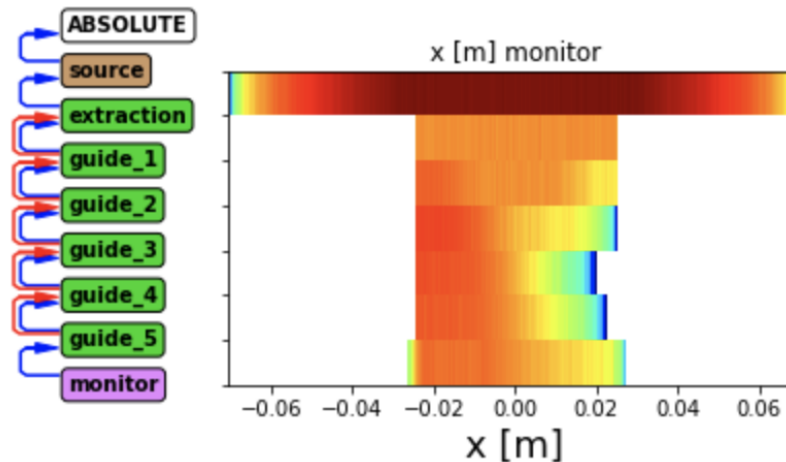
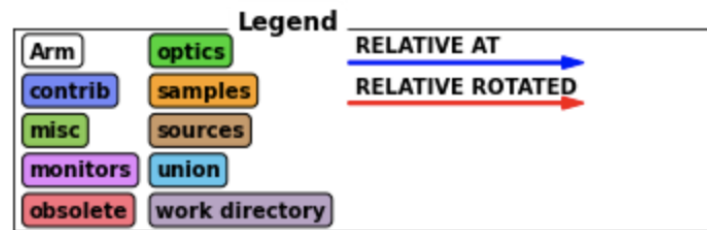


- Filename pitfall

```
mon = instrument.add_component("monitor", "PSD_monitor")
mon.set_parameters(nx = 100, ny = 100, filename = "psd.dat",
                  xwidth = 0.08, yheight = 0.08, restore_neutron = 1)
mon.set_AT([0,0,0.1 + guide.l], RELATIVE=guide)
```

- Diagrams

```
instrument.show_diagram(analysis=True, variable="x")
```



McStasScript

Demo



- Visualization

```
instrument.show_instrument()
```

ncount=300 --dir=demo_instrument_mcdisplay_1 wavelength=5.0

Previous

Pause

Next

Ray index 82 / 299

Keep rays ☐

Scatter Markers ☒

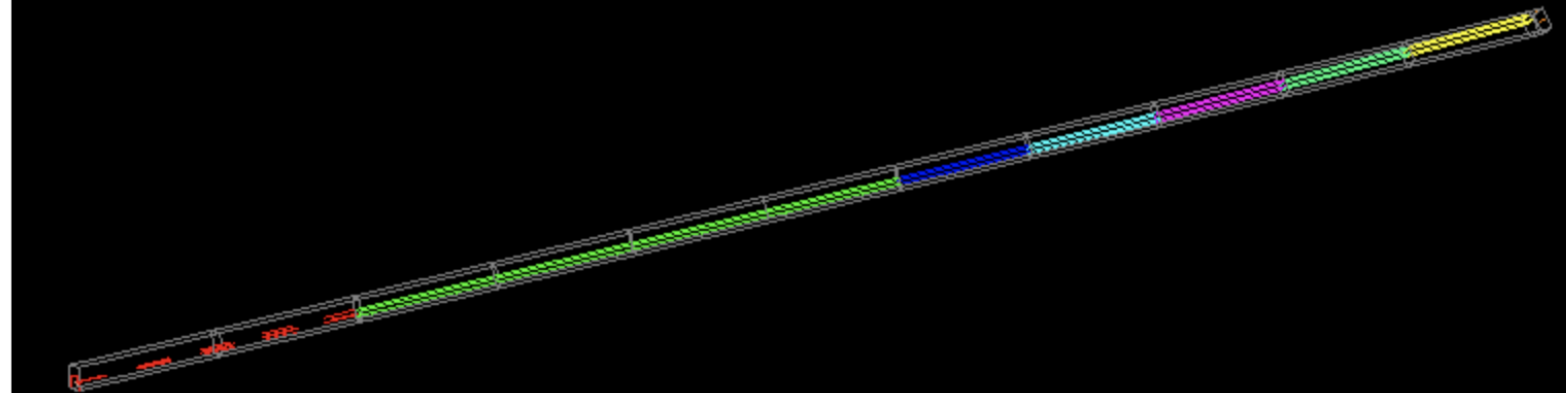
Reset view:

Home

Side

Top

Show BB ☒



McStasScript

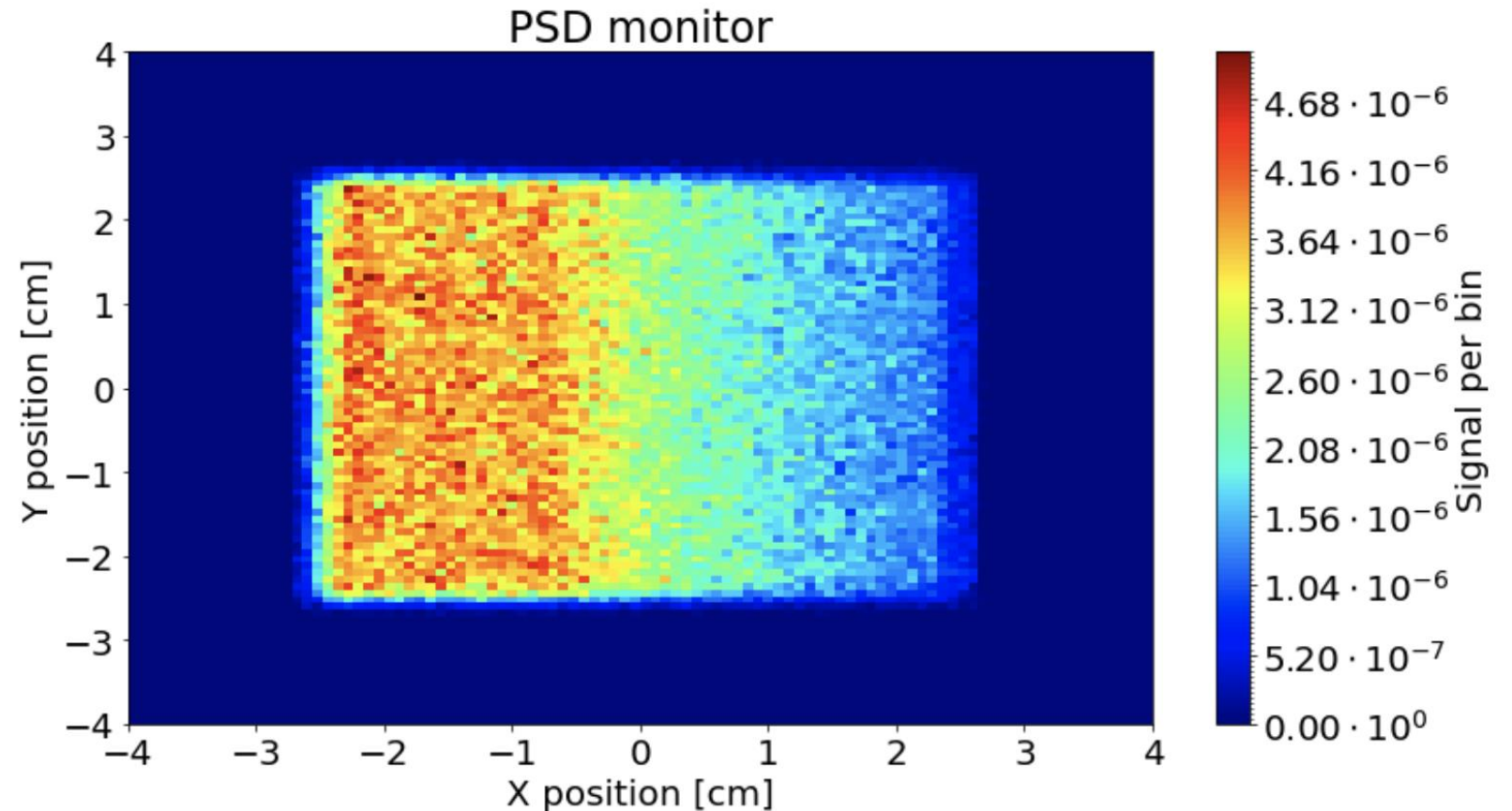
Demo



- Settings
- Parameters
- Run simulation
- Plot data

```
instrument.settings(ncount=1E6)  
instrument.set_parameters(wavelength=5.0)  
data = instrument.backengine()
```

```
ms.make_plot(data, fontsize=20)
```



McStasScript

Demo



- Python objects

```
data
```

```
[  
  McStasData: monitor type: 2D  I:0.0106059 E:1.95751e-05 N:298700.0]
```

- Metadata

```
monitor = ms.name_search("monitor", data)  
print(monitor.metadata)
```

```
metadata object  
component_name: monitor  
filename: psd.dat  
2D data of dimension (100, 100)  
  [-4.0: 4.0] X position [cm]  
  [-4.0: 4.0] Y position [cm]  
Signal per bin  
Instrument parameters:  
  wavelength = 5.0
```

- Numpy arrays

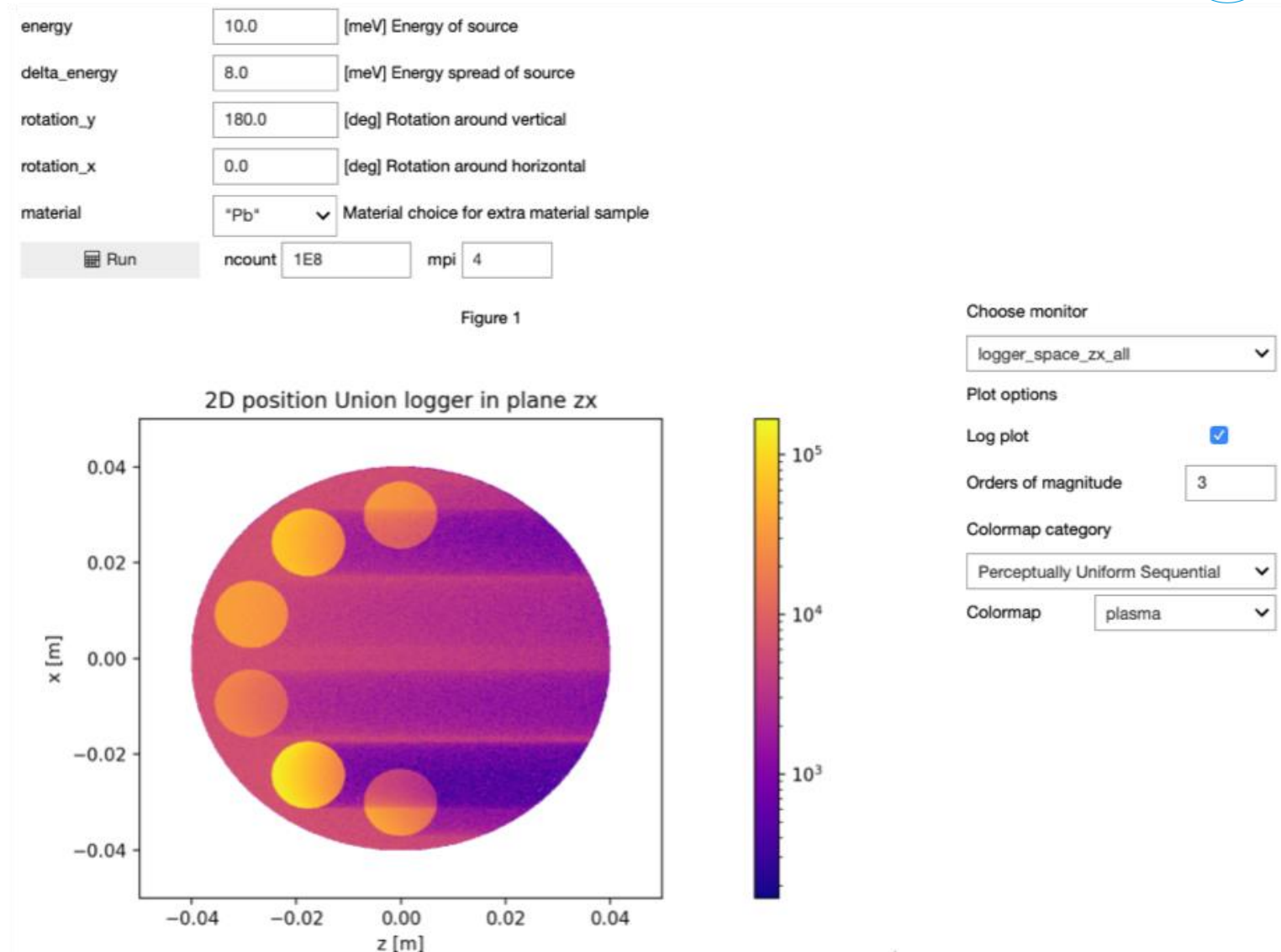
```
monitor.Intensity[20:28, 40:44]
```

```
array([[3.23134550e-06, 3.14453425e-06, 2.93890316e-06, 3.20877876e-06],  
       [3.38387710e-06, 3.33175403e-06, 3.50741480e-06, 3.58799402e-06],  
       [3.55206636e-06, 4.25656257e-06, 3.73936568e-06, 3.76360681e-06],  
       [3.87306679e-06, 2.86940652e-06, 3.28559204e-06, 3.16032605e-06],  
       [4.09563983e-06, 3.77835214e-06, 3.47772083e-06, 2.82467245e-06],  
       [3.86452494e-06, 3.40783311e-06, 3.92903001e-06, 4.12164357e-06],  
       [3.20800343e-06, 3.72103256e-06, 3.26505458e-06, 3.65186330e-06],  
       [3.18478308e-06, 3.86097677e-06, 3.37923228e-06, 3.49790361e-06]])
```

McStasScript

Demo

- Simulation widget

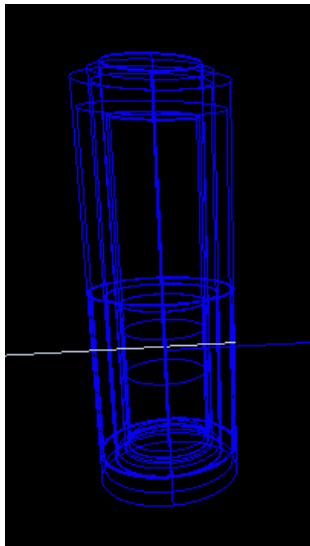


McStasScript



Tools

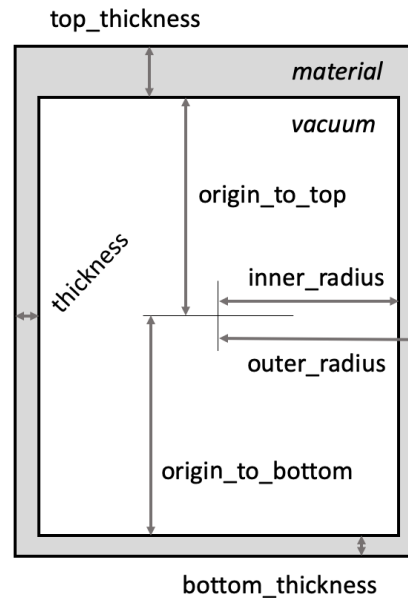
- Cryostat tool
- NCrystal tool



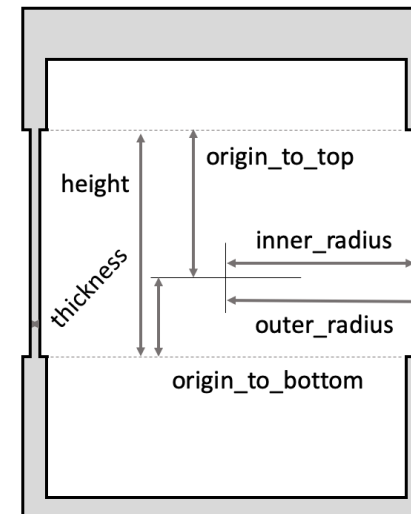
```
cryostat = ms.Cryostat("orange", instrument)
cryostat.set_AT([0,0,2], source)

cryostat.add_layer(inner_radius=70E-3/2, outer_radius=75E-3/2,
                  origin_to_top=200E-3, top_thickness=-1E-3,
                  origin_to_bottom=83E-3, bottom_thickness=5E-3,
                  material="Al", p_interact=0.2)
cryostat.last_layer.add_window(outer_radius=73E-3/2, origin_to_top=44.42E-3, origin_to_bottom=88.2E-3)
cryostat.last_layer.add_window(inner_radius=71E-3/2, height=40E-3)
```

Cryostat layer



Window



McStasScript

Documentation

- Comprehensive documentation
- Descriptions of the classes
- Tutorials for
 - McStasScript
 - McStas features
 - McStas Union components
- <https://mads-bertelsen.github.io>
 - Should be moved to better place
- Workshop / School available

The screenshot displays the McStasScript documentation website. The left sidebar contains a navigation menu with sections: GETTING STARTED (Overview, Installation, Version history, Quick start), USER GUIDE (Instrument object, Component object, Parameters and variables, Data, Plotting, Functions, Widgets, Instrument reader), MCSTASSCRIPT TUTORIAL (McStasScript introduction, Advanced McStas features: SPLIT, EXTEND and WHEN, JUMP, Dynamic instrument cuts with MCPL bridges, Diagnostics), and MCSTAS UNION TUTORIAL (The Union components, Advanced geometry using the Union components, Visualizing what happens in Union master). The main content area is titled 'Quick start' and includes a paragraph about the quick start guide, followed by sections for 'Importing the package' (showing `import mcstas as ms`), 'Creating the first instrument object' (showing `instrument = ms.McStas_instr("first_instrument")`), and 'Finding a component' (showing `instrument.available_components()` and a list of available component categories: contrib, misc, monitors, obsolete, optics, samples, sources, union). A right sidebar shows a 'Contents' table of contents.

McStasScript documentation

Search the docs ...

GETTING STARTED

- Overview
- Installation
- Version history
- Quick start

USER GUIDE

- Instrument object
- Component object
- Parameters and variables
- Data
- Plotting
- Functions
- Widgets
- Instrument reader

MCSTASSCRIPT TUTORIAL

- McStasScript introduction
- Advanced McStas features: SPLIT
- Advanced McStas features: EXTEND and WHEN
- Advanced McStas features: JUMP
- Dynamic instrument cuts with MCPL bridges
- Diagnostics

MCSTAS UNION TUTORIAL

- The Union components
- Advanced geometry using the Union components
- Visualizing what happens in Union master

Quick start

This section is a quick start guide that will show the basic functionality of McStasScript. It assumes the user is already familiar with McStas itself, if this is not the case, it is recommended to start with the tutorial which can serve as an introduction to both McStas and McStasScript.

Importing the package

McStasScript needs to be imported into the users python environment.

```
import mcstas as ms
```

Creating the first instrument object

Now the package can be used. Start with creating a new instrument, just needs a name. For a McXtrace instrument use McXtrace_instr instead.

```
instrument = ms.McStas_instr("first_instrument")
```

Finding a component

The instrument object loads the available McStas components, so it can show these in order to help the user.

```
instrument.available_components()
```

Here are the available component categories:

- contrib
- misc
- monitors
- obsolete
- optics
- samples
- sources
- union

Call `available_components(category_name)` to display

```
instrument.available_components("sources")
```

Here are all components in the sources category.

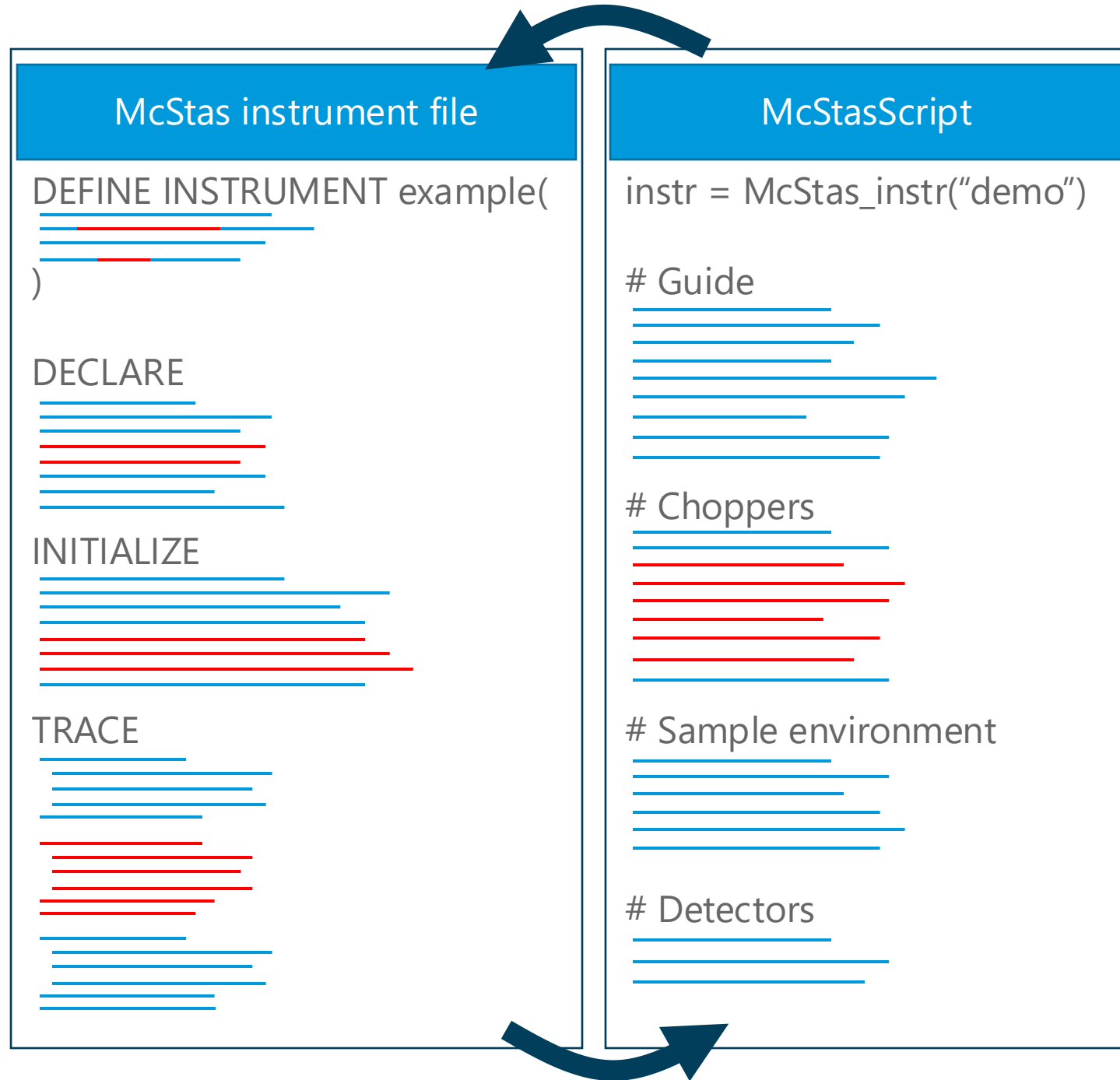
Adapt_check	Moderator	Source_Optimizer	Source_gen
ESS_butterfly	Monitor_Optimizer	Source_adapt	Source_simple

Contents

- Importing the package
- Creating the first instrument object
 - Finding a component
 - Adding the first component
 - Set parameters
 - Instrument parameters
 - Inserting a sample component
 - Adding a monitor
- Setting up the simulation
 - Performing the simulation
- Plot the data
 - Access the data

Code structure

- ## mcstas-pygen translator





McStasScript

Installation

- Install from pip / conda
 - **pip install McStasScript**
 - **conda install -c conda-forge mcstasscript**
 - Included as dependency if you install McStas, e.g.
conda install -c conda-forge mcstas
- Already available in your VISA instance in ESS conda environment

McStasScript

Exercise

- McStasScript quiz

- Questions that need to be answered by a number

```
quiz.question_1(12)
```

Incorrect

- Multiple choice questions need a string, options typically A, B, C, D, ...

```
quiz.question_3("D")
```

Not recognized

This multiple choice question did not contain that option!

- Instrument questions check you have set up an instrument correctly

```
quiz.question_7(instrument)
```

Correct!

Copy entire exercise and cryostat_demo folder to your environment
Then quit your VISA instance and open as Jupyter Lab



*Also cryostat_demo notebook if
you are curious! Uses NCrystal*